

ESCUELA COLOMBIANA DE INGENIERÍA PROGRAMACIÓN ORIENTADA A OBJETOS

S11: 2025-02

G01

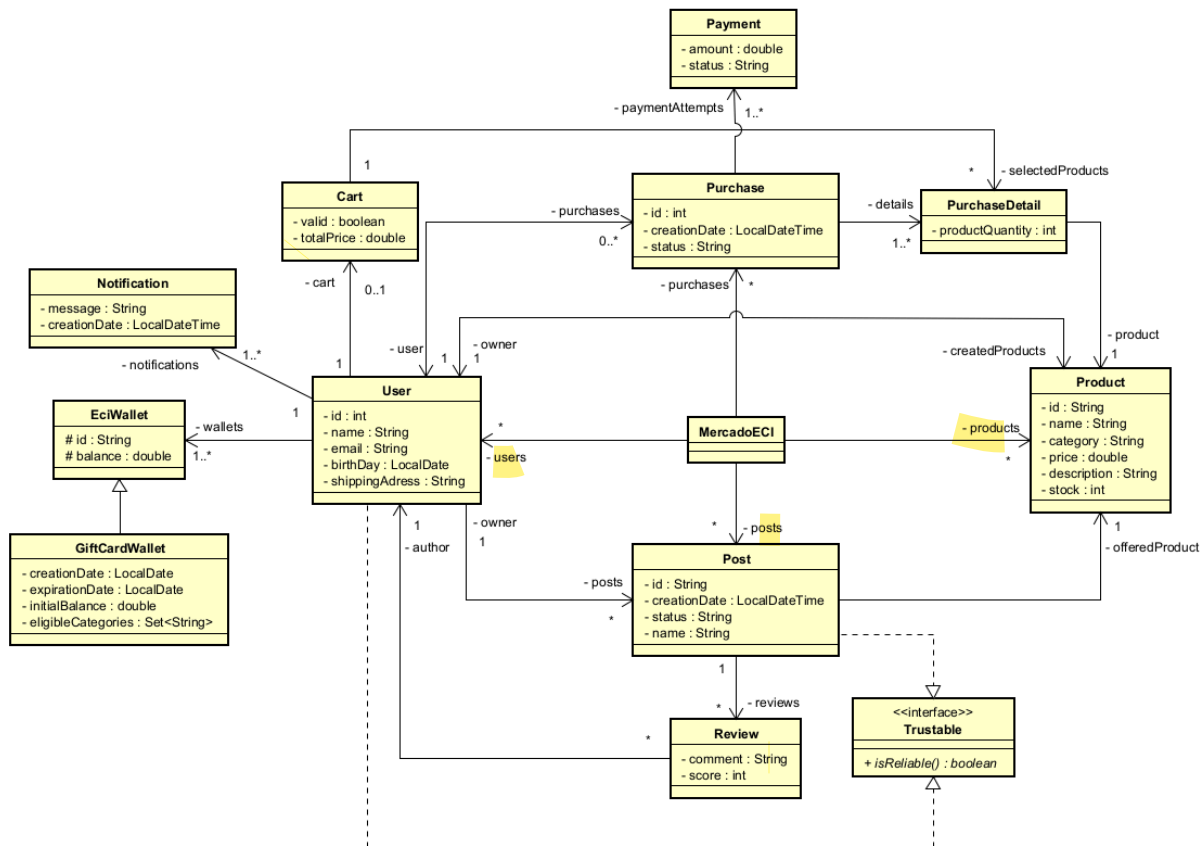
+Parcial segundo tercio. Nota esperada: ____

MercadoECI – Comercio Electrónico para Estudiantes

MercadoECI ha sido un sistema de comercio electrónico acogido por estudiantes y por la comunidad académica en general. Debido a esto, las directivas de la ESCUELA han solicitado implementar algunas funcionalidades de seguridad.

Nuevos requisitos funcionales:

- **GiftCardWallet:** Actualmente, MercadoECI desea incorporar una nueva modalidad de pago que permita a los usuarios utilizar bonos con saldo predefinido. Para esto, se requiere incluir en el sistema:
 - Un nuevo tipo de wallet denominado **GiftCardWallet**, el cual representa un bono electrónico con un saldo limitado, una fecha de expiración y restricciones de uso según las categorías de productos permitidas.
 - Cada GiftCard deberá registrar la fecha de emisión, la fecha de expiración, el saldo inicial y disponible, y la lista de categorías elegibles en las cuales puede utilizarse.
 - Para poder realizar un pago (checkout un carrito) utilizando este tipo de wallet, el sistema debe verificar:
 - Que el balance sea mayor o igual al valor de la compra - *condicion que se aplica para todos los tipos*
 - Que la fecha de expiración de la GiftCard sea posterior a la fecha actual.
 - Que el carrito de compras contenga únicamente productos pertenecientes a las categorías permitidas por la GiftCard.
- **Confiabilidad:**
 - **Usuario:** Se considera no confiable si la mayoría de sus compras tiene el estado “denegado” o si alguno de sus posts no es confiable.
 - **Post:** Se considera no confiable si el promedio de las calificaciones de sus reseñas es menor o igual a 2.0, si permanece activo cuando el producto asociado no tiene stock, o si posee más de una reseña escrita por el mismo usuario.



(Todos los contenedores son ArrayList, excepto los Usuarios, Posts y Productos que son HashMap<K,V> siendo K: el ID de c/u)

I. (30%) IMPLEMENTACIÓN

MDD

1. Estudie la especificación (documentación + encabezado) del método y los diagramas de secuencia.
2. Actualice el diagrama de clase con los nuevos elementos.
3. Implemente la clase responsable inicial (solo atributos).
4. Implemente la clase responsable de las excepciones del sistema
5. Implemente los métodos correspondientes al diseño. Incluya la documentación (si es necesario).

En MercadoECI

public boolean canCheckout(int userId, String walletId)

Verifica si el carrito de compras de un usuario puede ser pagado utilizando una wallet específica.

Si por alguna razón la wallet no puede ser utilizada para realizar el pago, se añadirá una notificación al usuario indicando el motivo del rechazo.

Parámetros:

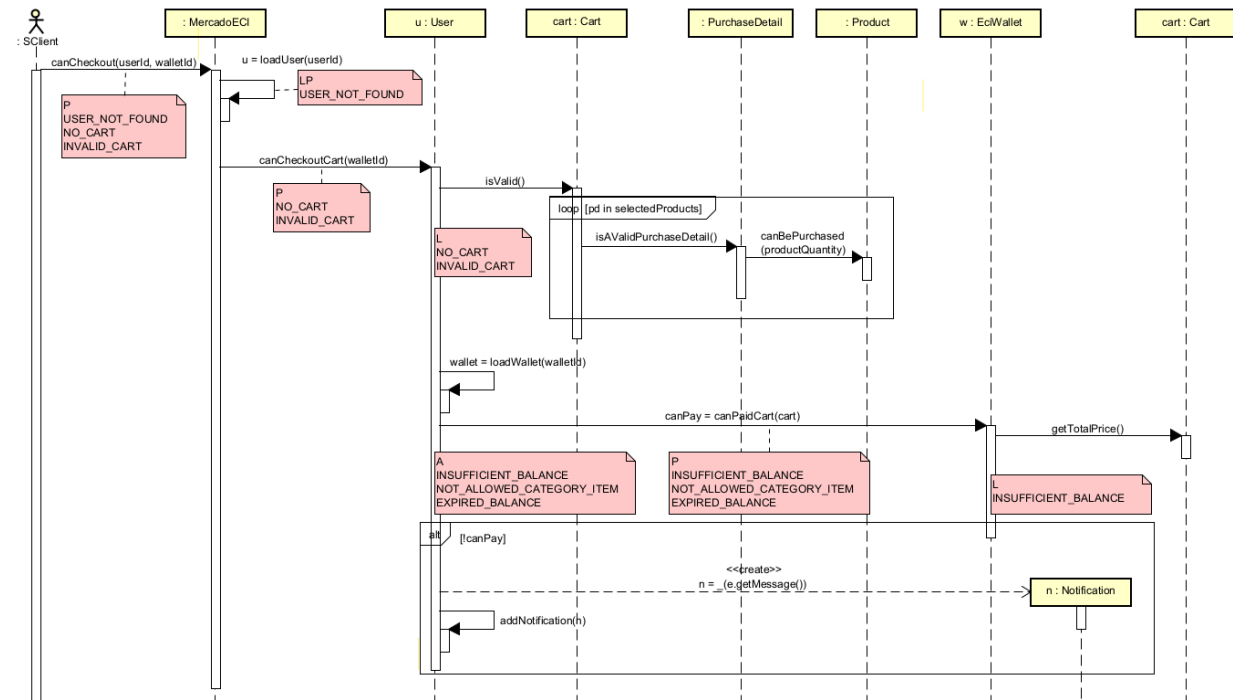
- **userId**: Identificador del usuario cuyo carrito se desea validar para el proceso de pago.
- **walletId**: Identificador de la wallet con la que se desea realizar el pago del carrito.

Throws:

MercadoECIException - USER_NOT_FOUND: El usuario no existe en el sistema.

MercadoECIException - NO_CART: El usuario no posee un carrito de compras que pueda ser validado.

MercadoECIException - INVALID_CART: El carrito contiene al menos un producto con una cantidad solicitada superior al stock disponible.



II. (25%) DISEÑANDO

Diseñe los métodos necesarios para cumplir con los compromisos asociados a la interfaz **Trustable** para los usuarios y posts. No olvide el manejo de excepciones.

1. Especifique (documentación + encabezado) el método propuesto
2. Realice los diagramas de secuencia correspondiente al diseño completo
3. Actualice el diagrama de clase con los nuevos elementos

III. (25%) EXTENDIENDO

MercadoECI desea incluir nuevos tipos de productos:

- **Físicos**: Estos productos poseen un peso en kilogramos y un costo de envío fijo, ya que solo estarán disponibles para usuarios dentro de un área de cobertura determinada de máximo 60km a la redonda.
 - Además de los productos físicos, se incluyen los productos perecederos, los cuales requieren condiciones adicionales de control. Cada producto perecedero debe registrar la fecha de expiración, la fecha desde la cual se encuentra en refrigeración, y solo podrá ser vendido si ha permanecido refrigerado por al menos 5 días
- **Digitales**: Este tipo de productos poseen stock ilimitado (por lo que siempre que el producto este activo se va a poder vender), una URL de descarga y el tamaño del archivo o software

1. Realice las extensiones necesarias en el diagrama de clases.
2. Implemente dichas extensiones (encabezados y atributos de los nuevos componentes)
3. Explique los cambios necesarios en los diseños anteriores para esta extensión.
4. Considerando el segundo principio SOLID. ¿Se cumplen en estos diseños?

IV. (20%) CONCEPTOS

1. ¿En qué condiciones es más apropiado utilizar una clase abstracta y en cuáles resulta mejor optar por una interfaz?
2. ¿Qué beneficios aporta manejar excepciones específicas (ej *MercadoEciException*) en lugar de capturar directamente la clase genérica *Exception* en Java?